Vídeo 3.3: Alterando e excluindo (PUT e DELETE)

Duração-alvo: 7 min | **Conceitos:** PUT, DELETE e código 204 | **O aluno sai sabendo:** corrigir e remover um livro pela API.

Antes de gravar: partir do código do vídeo 3.2 e deixar o requisicoes.http aberto.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “Já sabemos consultar e cadastrar. Faltam duas ações: corrigir um livro que foi cadastrado com erro e tirar da biblioteca um livro que não existe mais. São o PUT e o DELETE.”

[0:30] Os dois verbos

Na tela: duas ilustrações: uma ficha corrigida substituindo a antiga e uma ficha sendo retirada do fichário.

Fala: “PUT quer dizer substituir. Na biblioteca: a ficha de um livro tem um erro de digitação, você entrega a ficha corrigida, inteira, e ela ocupa o lugar da antiga. Por isso o PUT leva o endereço do livro, barra livros barra dois, e a ficha completa no corpo. DELETE quer dizer excluir. Aqui não precisa de corpo: o endereço já diz qual livro sai.”

[1:30] Escrevendo o PUT

Na tela: escrever a rota abaixo da rota POST, com o cursor acompanhando cada trecho.

```
app.put('/livros/:id', (req, res) => {
  const id = Number(req.params.id);
  const livro = livros.find((l) => l.id === id);

  if (!livro) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  const { titulo, autor } = req.body;
  livro.titulo = titulo;
  livro.autor = autor;

  res.json(livro);
});
```

Fala: “A rota começa igual à de buscar um livro: pega o id do endereço e procura o livro. Se não existir, 404. Se existir, pegamos título e autor do corpo e trocamos os valores da ficha. Por fim, respondemos com o livro já alterado. Atenção: como o PUT substitui a ficha inteira, o cliente precisa mandar todos os campos. Hoje, se faltar algum, ele fica sem valor. No módulo 5 vamos aprender a impedir isso.”

[3:15] Escrevendo o DELETE

Na tela: escrever a rota logo abaixo.

```
app.delete('/livros/:id', (req, res) => {
  const id = Number(req.params.id);
  const posicao = livros.findIndex((l) => l.id === id);

  if (posicao === -1) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  livros.splice(posicao, 1);
  res.status(204).send();
});
```

Fala: “Para excluir, procuramos a posição do livro na lista com findIndex. Se não achar, ele devolve menos um, e respondemos 404. Se achar, splice remove da lista o livro daquela posição. O um diz: remova só um. E respondemos com 204, sem conteúdo. 204 quer dizer: deu certo, e não tenho nada para te devolver. Como o livro foi excluído, realmente não há nada para mostrar.”

[4:45] Testando

Na tela: no requisicoes.http, escrever os três pedidos abaixo, reiniciar o servidor e enviar um de cada vez, conferindo a lista com o GET entre eles.

```
PUT http://localhost:3000/livros/1
 Content-Type: application/json

 {
   "titulo": "Dom Casmurro",
   "autor": "Machado de Assis (edição revisada)"
 }

 ###

 DELETE http://localhost:3000/livros/2

 ###

 DELETE http://localhost:3000/livros/99
```

Fala: “Hora de testar. Reinicio o servidor. Faço um PUT no livro 1, alterando o autor: 200 e a ficha corrigida. Consulto a lista, e a mudança está lá. Agora um DELETE no livro 2: 204 No Content. Consulto a lista: o livro 2 sumiu. E por fim um DELETE no livro 99, que não existe: 404. Cada resposta contou direitinho o que aconteceu.”

[6:20] Resumo

Na tela: duas linhas: PUT substitui a ficha inteira; DELETE exclui e responde 204.

Fala: “Resumindo: o PUT substitui a ficha inteira, e por isso precisa de todos os campos. O DELETE exclui, e responde 204, sem conteúdo. Nos dois, um livro que não existe resulta em 404. No próximo vídeo vamos juntar tudo e dar nome a isso.”

Exercício sugerido: alterar o título de um livro com PUT e excluir outro com DELETE, conferindo a lista depois de cada pedido.